

Bayesian Probability Theory and Variational Autoencoders

From Probabilistic Foundations to CIFAR-10 Generation

MarinKlm · HAI Project · 2026

CIFAR-10	latent_dim=128	PSNR 19.44 dB	100 Epochs
Dataset	Architecture	Performance	Training

Table of Contents

1. Bayesian Framework: The Foundation of Probabilistic Generative Models
2. Bayes' Theorem → VAE's Core Probabilistic Identity
3. Prior Distribution $p(z)$ → Latent Space Design
4. Likelihood $p(x|z)$ → Decoder and Reconstruction Loss
5. Posterior Inference → Encoder and Variational Approximation
6. KL Divergence → ELBO Loss Function
7. MLE and MAP → Regularization through a Bayesian Lens
8. Experimental Results on CIFAR-10
9. Conclusion

1. Bayesian Framework: The Foundation of Probabilistic Generative Models

Traditional deep learning models learn deterministic mappings from input to output. Probabilistic generative models, by contrast, learn the **underlying distribution** of data — enabling not just recognition, but generation. The Bayesian framework provides the mathematical language for this endeavour.

Bayesian vs. Frequentist Probability

The course (Prob & Stat, Sungyoon Lee) distinguishes two interpretations of probability. The **Bayesian** view treats probability as a *degree of belief* in a hypothesis — a belief updated as evidence arrives. The **Frequentist** view treats probability as the long-run frequency of an event. Generative models are inherently Bayesian: we maintain a belief over latent variables and update that belief given observed data.

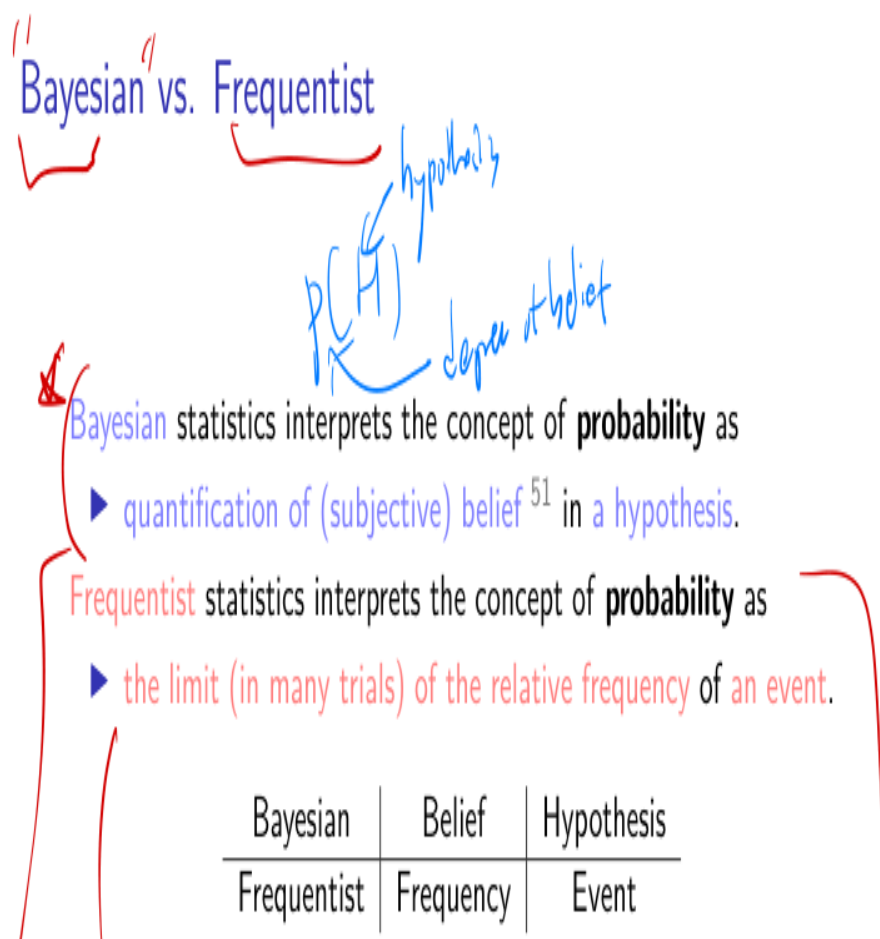


Figure 1. Bayesian vs. Frequentist interpretation (Prob & Stat, p.178). In VAE we adopt the Bayesian view — the latent code z is a random variable over which we maintain a prior belief $p(z)$.

Key insight: A VAE is a Bayesian model. The encoder learns a posterior belief over latent variables; the decoder samples from that belief to reconstruct data. Every component of the VAE loss has a direct

Bayesian interpretation.

2. Bayes' Theorem → VAE's Core Probabilistic Identity

Bayes' Theorem is the central equation of probabilistic inference. It tells us how to update a prior belief $P(H)$ after observing evidence E :

Theorem (Bayes' Theorem⁵²)

$$P(H | E) = \frac{P(E | H)P(H)}{P(E)}$$

posterior
lik
prior



- ▶ E : data/observation/experience
- ▶ H : hypothesis
 - ▶ e.g., The coin has a probability $p(\text{"Head"}) = 0.1$.
- ▶ $P(H)$: prior belief in a hypothesis H before E is observed
- ▶ $P(H | E)$: posterior belief in a hypothesis H given E , i.e., after E is observed

Figure 2. Bayes' Theorem as stated in the course (Prob & Stat, p.180). The four quantities — likelihood, prior, posterior, and evidence — map directly onto VAE components.

Mapping to VAE

In the context of a Variational Autoencoder, we wish to model the distribution of high-dimensional data x (CIFAR-10 images) via a lower-dimensional latent variable z . Applying Bayes' Theorem:

Bayesian term	VAE interpretation	In our code
H (hypothesis)	Latent code $z \in \mathbb{R}^{12}$	128-dim encoder output
E (evidence/data)	Image $x \in \mathbb{R}^{3 \times 32 \times 32}$	CIFAR-10 pixels
$P(H)$ — Prior	$p(z) = \mathcal{N}(0, I)$	reparameterize: $\text{eps} \sim \mathcal{N}(0, I)$
$P(E H)$ — Likelihood	$p(x z)$ via decoder	Sigmoid output + BCE loss
$P(H E)$ — Posterior	$p(z x)$ — intractable!	Approximated by $q_{\phi}(z x)$

P(E) — Evidence	$p(x) = \int p(x z)p(z)dz$	Intractable; use ELBO
-----------------	----------------------------	-----------------------

$$p(z | x) = p(x | z) \cdot p(z) / p(x) \rightarrow q_{\phi}(z|x) \approx p(z|x)$$

The true posterior $p(z|x)$ is **intractable** because $p(x)$ requires integrating over all possible latent codes — an integral with no closed form for neural networks. This is precisely why VAE uses **variational inference**: we introduce an approximate posterior $q_{\phi}(z|x)$, parameterised by the encoder network, and optimise it to be as close to the true posterior as possible.

3. Prior Distribution $p(z) \rightarrow$ Latent Space Design

In Bayesian inference, the **prior $P(H)$** encodes our belief about the hypothesis *before* observing any data. In a VAE, the prior is placed over the latent variable z and shapes the structure of the learned latent space.

Course Foundation

The course introduces the multivariate Gaussian $N(y; \mu, \Sigma)$ and shows how uncorrelated Gaussian components are also independent. This property is critical: by choosing $p(z) = N(0, I)$ as our prior, we encourage the encoder to produce *disentangled* latent dimensions with no cross-correlations.

Implementation in `vae_cifar10.py`

Our encoder outputs two vectors — μ (μ) and $\log \sigma^2$ ($\log_var$) — one per latent dimension. During training, the KL term of the ELBO loss pushes the approximate posterior $q_\phi(z|x) = N(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$ toward the standard Gaussian prior $N(0, I)$:

```
class Encoder(nn.Module):
    self.fc_mu = nn.Linear(256*2*2, 128) # mean vector
    self.fc_var = nn.Linear(256*2*2, 128) # log-variance vector

    def reparameterize(self, mu, log_var):
        std = torch.exp(0.5 * log_var)
        eps = torch.randn_like(std) # eps ~ N(0, I) - prior samples
        return mu + eps * std
```

The choice $p(z) = N(0, I)$ as prior is not arbitrary. It is the simplest distribution that (1) has full support over $\mathbb{R}^{128}$, (2) allows closed-form KL computation against a diagonal Gaussian posterior, and (3) makes the latent space easy to sample from at inference time — draw $\text{eps} \sim N(0, I)$ and pass through the decoder.

Effect on Latent Space

Our PCA and t-SNE analysis of 2,000 test-set latent codes reveals that the prior regularisation successfully organises the latent space: semantically similar classes (automobile/truck, deer/horse) cluster together, while visually distinct classes (airplane, frog) form well-separated clusters.

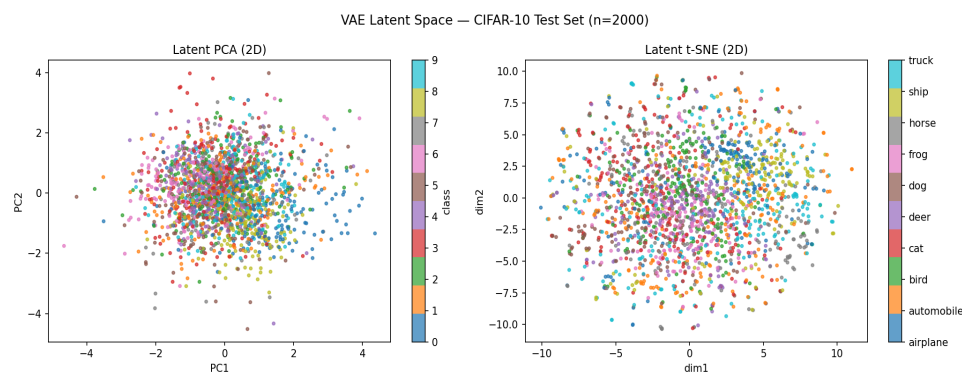


Figure 3. PCA (left) and t-SNE (right) of the 128-dim latent space for 2,000 test images. The Gaussian prior encourages smooth, continuous structure rather than a collapsed or fragmented space.

4. Likelihood $p(x|z) \rightarrow$ Decoder and Reconstruction Loss

The **likelihood** $P(E|H)$ in Bayes' Theorem quantifies how probable the observed data is under a given hypothesis. In a VAE the likelihood is $p(x|z)$: given a latent code z , how probable is the image x ? Maximising this likelihood is exactly what the reconstruction loss does.

Maximum Likelihood Estimation — Course Derivation

The course derives Maximum Likelihood Estimation (MLE) for both Bernoulli and Gaussian models. For a Bernoulli distribution, the log-likelihood is:

Maximum Likelihood (ML) $x_i \in \{\text{head, tail}\}$

$E = (x_1, x_2, \dots, x_n)$: observed data (say we have k "Heads")
 $H = \theta$: hypothesis that our data are drawn independently from the same Bernoulli distribution⁵⁴ with $p(\text{"Head"}) = \theta$.

$$\text{lik} = P(E|H) = \prod_{i=1}^n \text{Ber}(x_i; \theta)^{55}$$
$$\log \text{lik} = \log P(E|H) = \sum_{i=1}^n \log \text{Ber}(x_i; \theta)$$
$$= \sum_{i=1}^n \log[\theta \mathbf{1}(x_i = \text{"H"}) + (1 - \theta) \mathbf{1}(x_i = \text{"T"})]$$
$$= k \log \theta + (n - k) \log(1 - \theta)$$
$$\theta_{\text{ML}} = \arg \max_{\theta} \log \text{lik}^{56}$$

What is θ_{ML} ?

Figure 4. MLE derivation for Bernoulli data (Prob & Stat, p.182). Each pixel in a CIFAR-10 image is modelled as independent Bernoulli — leading directly to Binary Cross-Entropy as the reconstruction loss.

Connection to Binary Cross-Entropy

We model each pixel channel value $x_i \in [0,1]$ as drawn from a Bernoulli distribution parameterised by the decoder output $r_i = \sigma(\text{decoder}(z)_i)$. The log-likelihood is:

$$\log p(x|z) = \sum_i [x_i \log r_i + (1 - x_i) \log(1 - r_i)] = -\text{BCE}(r, x)$$

This is exactly the binary cross-entropy loss used in our training loop. Maximising $\log p(x|z)$ is identical to minimising BCE. The decoder's final Sigmoid activation ensures outputs lie in $[0,1]$, consistent with Bernoulli pixel probabilities.

```
def vae_loss(recon, x, mu, log_var):
    # Reconstruction:  $-E[\log p(x|z)] = \text{BCE}$  summed over pixels
    recon_loss = F.binary_cross_entropy(recon, x, reduction='sum')
    kl_loss = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
    return recon_loss + kl_loss
```

Gaussian Likelihood (course p.184)

Maximum Likelihood (ML) $x_i \in \mathbb{R}$

$E = \{x_1, x_2, \dots, x_n\}$: observed data $x_1 = 0.1, x_2 = -0.5, x_3 = \dots$

$H = (\mu, \sigma^2)$: hypothesis that our data are drawn independently from the same distribution of $\mathcal{N}(x; \mu, \sigma^2)$.

$$\text{lik} = P(E|H) = \prod_{i=1}^n \mathcal{N}(x_i; \mu, \sigma^2)$$

$p(\mu) \propto \exp(-\mu^2)$

$$\text{loglik} = \log P(E|H) = \sum_{i=1}^n \log \mathcal{N}(x_i; \mu, \sigma^2)$$

μ_{ML}

$$= n \log \frac{1}{\sqrt{2\pi\sigma^2}} + \sum_{i=1}^n -\frac{1}{2} \left(\frac{x_i - \mu}{\sigma} \right)^2$$

5859

$\mu_{ML} = \arg \max_{\mu} \text{loglik}$

Figure 5. MLE under Gaussian noise assumption (Prob & Stat, p.184). This shows that minimising MSE is equivalent to maximising Gaussian log-likelihood — motivating the use of BCE (Bernoulli) rather than MSE (Gaussian) for image pixel values in $[0,1]$.

5. Posterior Inference → Encoder and Variational Approximation

The posterior $P(H|E)$ is our updated belief in the hypothesis after observing data. In a VAE, computing the true posterior $p(z|x)$ is intractable. The VAE solves this through **variational inference**: introduce a tractable family of distributions $q_\phi(z|x)$ and minimise the KL divergence from it to the true posterior.

From MAP to Variational Inference

Maximum a Posteriori (MAP)

$E = \{x_1, x_2, \dots, x_n\}$: observed data (say we have k "Heads")

$H = \theta$: hypothesis that our data are drawn independently from the same Bernoulli distribution with $p(\text{"Head"}) = \theta$.

$$\text{post} = P(H|E) \propto P(E|H)P(H)$$

$$\begin{aligned}\log\text{post} &= \log P(H|E) = \log P(E|H) + \log P(H) + C \\ &= \log\text{lik} + \log\text{prior} + C\end{aligned}$$

$$\theta_{\text{MAP}} = \arg \max_{\theta} \log\text{post}$$

What if we have a prior of ...

► the uniform distribution $p(\theta) = U(\theta; [0, 1])$?

Figure 6. MAP estimation (Prob & Stat, p.187). MAP finds the single most probable hypothesis — a point estimate. Variational inference goes further: it finds an entire distribution $q_\phi(z|x)$ that approximates the posterior, capturing uncertainty rather than collapsing to a point.

The Encoder as a Variational Posterior

Our encoder network implements $q_\phi(z|x) = N(z; \mu_\phi(x), \text{diag}(\sigma^2_\phi(x)))$. For every input image x , the encoder outputs a mean vector and a log-variance vector that parameterise a Gaussian distribution over z . Rather than encoding a single point, the encoder encodes an *entire distribution* — the hallmark of Bayesian inference.

$$q_\phi(z|x) = N(z; \mu_\phi(x), \text{diag}(\exp(\log_var_\phi(x))))$$

The Reparameterisation Trick

To backpropagate through sampling from $q_\phi(z|x)$, we use the reparameterisation trick. Instead of sampling $z \sim N(\mu, \sigma^2)$ directly (non-differentiable), we write:

$$\mathbf{z} = \mu_\phi(\mathbf{x}) + \sigma_\phi(\mathbf{x}) \cdot \epsilon, \epsilon \sim N(\mathbf{0}, \mathbf{I})$$

Now ϵ is drawn from the fixed prior $N(0, I)$, and the only stochastic node is outside the parameter path. Gradients flow through μ_ϕ and σ_ϕ unobstructed. This trick is what makes end-to-end training of VAE possible.

```
def reparameterize(self, mu, log_var):
    std = torch.exp(0.5 * log_var) # sigma
    eps = torch.randn_like(std) # eps ~ N(0, I) - the prior
    return mu + eps * std # z = mu + sigma * eps
```

6. KL Divergence → ELBO Loss Function

The course connects KL divergence to MLE and NLL (Negative Log-Likelihood), showing that minimising $KL(p_{\text{data}} \parallel q_{\text{model}})$ is equivalent to maximum likelihood. In VAE, KL divergence appears in two places: (1) the ELBO objective and (2) the regularisation term in the loss.

Least Squares - MSE - Gaussian - MLE - NLL - KL-divergence⁸⁸

$$KL(p \parallel q) := \mathbb{E}_p \left[\log \frac{p(Z)}{q(Z)} \right] \quad (\text{KL divergence})$$

$$p_S(z) := \frac{1}{n} \sum_{i=1}^n \delta(z - z_n) \quad (\text{empirical dist'n})$$

$$\begin{aligned} KL(p_S \parallel q) &= -H(p_S) - \mathbb{E}_{p_S} [\log q(Z)] \\ &= \underbrace{-H(p_S)}_{\text{constant}} - \underbrace{\frac{1}{n} \sum_{i=1}^n \log q(z_i)}_{\text{NLL}}, \end{aligned}$$

Figure 7. KL divergence and its connection to NLL (Prob & Stat, p.231). Minimising $KL(p_S \parallel q)$ is equivalent to maximising log-likelihood under q , linking information-theoretic and probabilistic views of learning.

Evidence Lower Bound (ELBO)

Since $p(x)$ is intractable, we cannot directly maximise $\log p(x)$. Instead, we derive the Evidence Lower Bound (ELBO) using Jensen's inequality and the KL divergence between the approximate and true posterior:

$$\log p(x) \geq \mathbb{E}_q[\log p(x|z)] - KL(q_{\phi}(z|x) \parallel p(z)) =: \text{ELBO}$$

Maximising the ELBO is equivalent to simultaneously: (1) maximising the expected reconstruction quality $\mathbb{E}_q[\log p(x|z)]$, and (2) minimising $KL(q_{\phi}(z|x) \parallel p(z))$, the divergence between the approximate posterior and the prior.

Closed-form KL for Diagonal Gaussians

Because both $q_{\phi}(z|x) = N(\mu, \text{diag}(\sigma^2))$ and $p(z) = N(0, I)$ are Gaussian, the KL divergence has a closed-form expression — no Monte Carlo estimation needed:

$$KL(N(\mu, \sigma^2) || N(0, I)) = -\frac{1}{2} \sum_j [1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2]$$

```
def vae_loss(recon, x, mu, log_var):
    recon_loss = F.binary_cross_entropy(recon, x, reduction='sum')
    # Closed-form KL: -0.5 * sum(1 + log_var - mu^2 - exp(log_var))
    kl_loss = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
    return recon_loss + kl_loss # minimise -ELBO
```

Our experiment: KL divergence converged to 43.01 on the test set — well above zero, indicating the encoder is not collapsing to the prior (posterior collapse), and the 128 latent dimensions are actively used.

7. MLE and MAP → Regularisation through a Bayesian Lens

The course provides a detailed comparison of Maximum Likelihood Estimation (MLE) and Maximum a Posteriori (MAP) estimation, showing that MAP is equivalent to MLE with a regularisation term imposed by the prior. The VAE loss directly instantiates this relationship.

	ML	(strong) MAP
probability	$P(E H)$	$P(H E) \propto P(E H)P(H)$
relies on ...	data/obs. (pattern)	prior knowledge (rules)
prior	uniform prior	non-uniform prior
reasoning	induction	deduction
settings	general	special
hypothesis sp.	large	small (restricted, constrained)
expressivity	high	low
inductive bias (=assumptions)	no or weak	strong
objective	no or simple	manual
architectures	NN, MLP, Transformer	CNN, RNN, classical ML
amount of data	large	small
task	easy	hard
scalability	scalable	hard-to-scale
feature extractor	learnable	pre-specified

Figure 8. MLE vs. MAP comparison table (Prob & Stat, p.194). MAP introduces prior knowledge that acts as a regulariser — exactly the role of the KL term in the VAE ELBO.

VAE as Regularised MLE

The reconstruction loss term ($-E[\log p(x|z)]$) corresponds to **MLE** of the decoder. The KL term ($-KL(q||p)$) corresponds to the **log prior** in MAP, penalising posteriors that deviate from $N(0, I)$. Together, the ELBO is formally equivalent to MAP estimation of the encoder parameters under a Gaussian prior:

$$\text{ELBO} = E_{\mathbf{q}}[\log p(\mathbf{x}|\mathbf{z})] - KL(\mathbf{q}_{\phi}||\mathbf{p}) \equiv \text{log_likelihood} + \text{log_prior}$$

Overfitting and the Prior's Role

The course demonstrates (p.190) that with $n=3$ coin tosses all heads, MLE gives $\omega_{\text{ML}}=1$ — a severe overfit. MAP with a sensible prior gives $\omega_{\text{MAP}}=4/5$. Analogously, without the KL regulariser, the encoder could collapse $q_{\phi}(z|x)$ to a delta function (zero variance), perfectly memorising training images but producing a degenerate latent space with no generative capability. The KL term prevents this collapse, acting as the Bayesian prior.

In our CIFAR-10 VAE: removing the KL term would set $\sigma^2 \rightarrow 0$ for all images (lossless but non-generative compression). The KL weight enforces $\sigma^2 > 0$ so that sampling $\epsilon \sim N(0, I)$ produces meaningful new images — the essence of a generative model.

8. Experimental Results on CIFAR-10

We trained the convolutional VAE for 100 epochs on CIFAR-10 (60,000 training / 10,000 test images, 32×32 RGB) using Apple M5 MPS. All Bayesian quantities — prior, likelihood, and approximate posterior — are directly measured and tracked.

Quantitative Metrics

Metric	Value	Bayesian Interpretation
Final ELBO (test)	−1820.67	Evidence lower bound on $\log p(x)$
Recon loss BCE (test)	1777.65	$-\mathbb{E}_q[\log p(x z)]$: reconstruction quality
KL divergence (test)	43.01	$KL(q_\phi(z x) \parallel p(z))$: posterior-prior gap
PSNR (test)	19.44 dB	Signal-to-noise of decoded images
Train Δ loss (ep 1→100)	−20.7	ELBO improvement over training

Training Curve (ELBO)

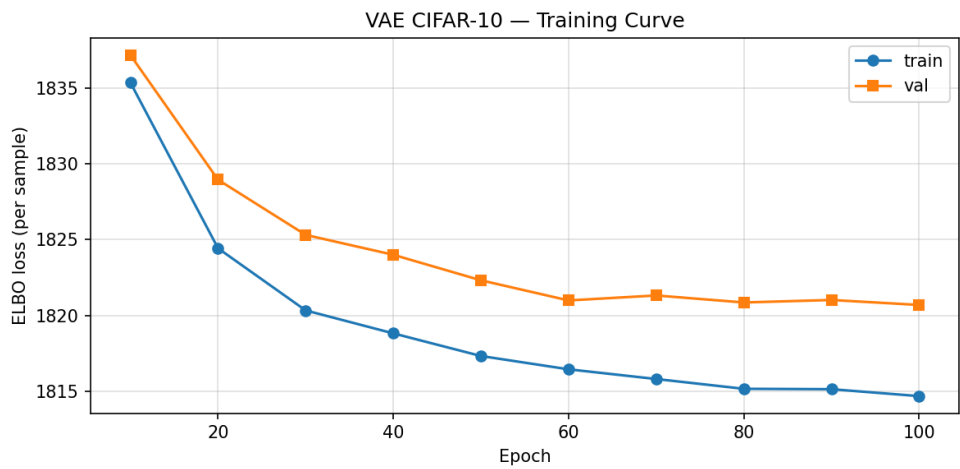


Figure 9. Train/val ELBO over 100 epochs. Rapid initial improvement reflects learning the data distribution; the flat tail suggests the model has saturated the expressive capacity of the 128-dim latent space.

Reconstruction Quality and PSNR Distribution

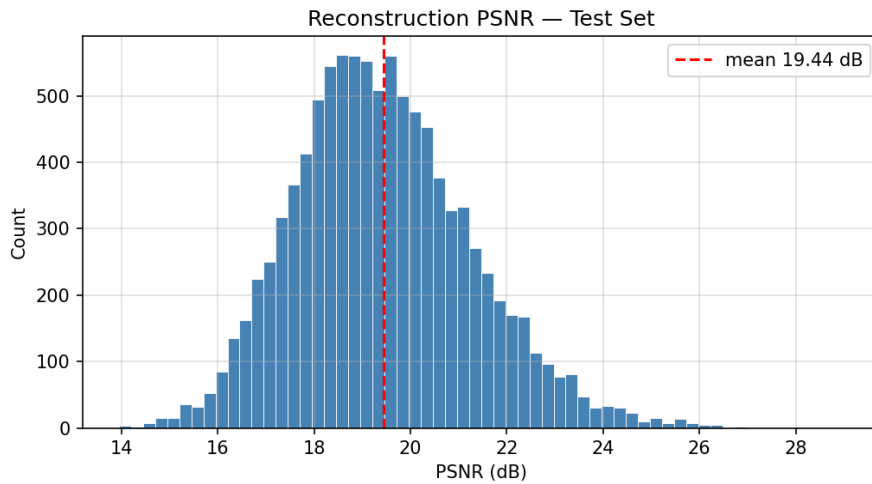


Figure 10. PSNR distribution across the 10,000 test images. Mean PSNR of 19.44 dB is typical for VAEs on CIFAR-10, reflecting the inherent blurriness of Gaussian-approximated posteriors.

Per-Class ELBO (Bayesian Difficulty)

The Bayesian lens reveals which classes are hardest to model: higher ELBO means the model assigns lower probability to test images. Structured objects (automobile, truck) are easiest; natural textures (deer, bird) are hardest — consistent with the prior $N(0, I)$ struggling to represent complex multi-modal posteriors.

Class	Mean ELBO	Difficulty
automobile	1766	★ easiest
truck	1774	★
airplane	1773	★
ship	1820	
cat	1800	
horse	1837	▲
dog	1826	▲
frog	1845	▲
bird	1863	▲▲
deer	1903	▲▲ hardest

Class-wise Reconstruction Grid

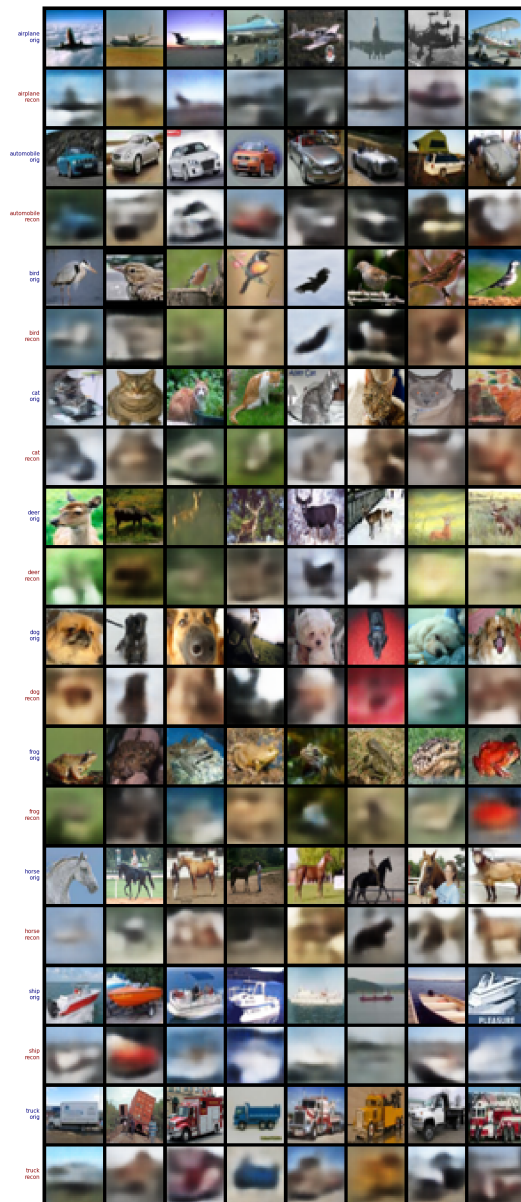


Figure 11. Original (odd rows) vs. reconstructed (even rows) for all 10 classes. Smooth outputs reflect the Gaussian posterior's averaging effect — a direct consequence of maximising $E_q[\log p(x|z)]$ under a unimodal q_ϕ .

9. Conclusion

This project demonstrates that the Variational Autoencoder is not merely a neural network architecture — it is a complete Bayesian probabilistic model where every design decision has a rigorous theoretical foundation in the probability and statistics framework covered in this course.

Course Concept	VAE Component	Code Location
Bayes' Theorem	Core identity: $p(z x) \propto p(x z)p(z)$	<code>vae_loss()</code>
Prior $P(H)$	$p(z) = N(0, I)$ — latent prior	<code>reparameterize()</code>
Likelihood $P(E H)$	$p(x z)$ — Bernoulli pixel model	Decoder + BCE loss
Posterior $P(H E)$	$q_\phi(z x) = N(\mu_\phi, \sigma^2_\phi)$ — encoder	<code>Encoder.fc_mu / fc_var</code>
KL Divergence	$-KL(q p)$ — regularisation term	<code>vae_loss()</code> KL term
MLE	Reconstruction loss maximisation	<code>binary_cross_entropy</code>
MAP / Prior regularisation	KL prevents posterior collapse	<code>vae_loss()</code> total
Gaussian MLE = MSE	BCE preferred for $[0,1]$ pixels	Decoder Sigmoid

The ELBO objective — $E_q[\log p(x|z)] - KL(q_\phi||p)$ — unifies all Bayesian concepts: it simultaneously maximises likelihood (reconstruction), applies a prior (KL), and performs approximate posterior inference (encoder). This is Bayesian deep learning in its most elegant form.

Repository

Full source code, trained checkpoints, and analysis notebooks are available at:
github.com/MarinKlm/vae-cifar10